23CSE455 Design Patterns

Case Study Report

Important note : remove the Grey part (placeholder) and replace your content.

Team: *[Exact team name, such as A1 or B1]*

Case study: *[Assigned domain, framework and title]*

Application: *[Selected real application and faculty approval]*

Date: *[YYYY-MM-DD]*

Submission repository: 23CSE455_DP_CaseStudy

Students

Roll number	Student name	Batch
Roll number	*Name*	*CSE A / EAC / AIDS /AIE*
Roll number	*Name*	*Batch*

Add: Add one row for each student.

1 Application and architecture

Original source: *[Repository URL, version/tag, exact baseline commit, access date, license]*

Environment: *[Language, framework, database and dependency versions]*

Scope: *[Files/modules inspected and excluded; reasons for exclusions]*

Architecture: *[Insert one diagram; explain components and a request/event flow]*

Run instructions: *[README path, setup/run/test commands and actual result]*

2 Part 1 Pattern discovery

Method: *[How candidates were found and source evidence checked]*

2.1 Totals at the baseline

Confirmed pattern types: *[Distinct names, such as Strategy and Observer; total number]*

Confirmed pattern instances: *[Independent collaborations; total number]*

Concrete participant classes: *[Classes implementing the variable roles; total number]*

Unique participant symbols: *[Distinct mapped classes/interfaces/functions; shared symbols counted once]*

Rejected and uncertain candidates: *[Separate counts and reasons; record observation scope]*

2.2 Pattern inventory

Pattern type	Instance names	Instances	Concrete classes
Strategy	Overdue Fine Calculation (example)	1	2
Observer	Book Return Notification (example)	1	1

Add: Replace the example rows with your findings. Record every identified candidate within the approved scope; no minimum pattern quota.

2.3 One record for each pattern instance

Instance name: *[Meaningful collaboration name, not just Strategy or a class name]*

Pattern type and category: *[Exact recognized name; GoF / GRASP / architecture / concurrency / framework idiom]*

Purpose and origin: *[Problem solved; application / framework / dependency / student-added]*

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Strategy	FinePolicy (example)	path:lines
ConcreteStrategy	RegularFine (example)	path:lines
ConcreteStrategy	StudentFine (example)	path:lines
Context	Library.returnBook (example)	path:lines

Observer example: Subject = not separately declared; ConcreteSubject = Library (also performs subject role); Observer = ReturnListener; ConcreteObserver = NotificationLog. Where code declares separate Subject/ConcreteSubject classes, name each actual class and corresponding method.

Add: Use actual qualified symbol names. A role may be a function or module; do not invent classes for non-class-based implementations. Include all participants, even across multiple files.

Evidence: *[Intent, relationships, calls/delegation and behavior; concise code excerpts]*

Diagram: *[Editable UML/component diagram and rendered image paths]*

Verification: *[Confirmed / rejected / uncertain; reviewer, justification, tests and alternatives]*

Add: Repeat the record for each confirmed instance. Record rejected/uncertain candidates with a reason. A class name, annotation or LLM assertion alone is insufficient.

3 Part 2 Evolution and refactoring

Add: Implement three realistic changes from your assigned case study. Repeat this record for CH01, CH02 and CH03.

Change and acceptance criteria: *[Requirement and observable expected results]*

Before and after: *[Exact commits and changed source paths]*

Implementation and tests: *[What changed; run command; actual test result and log path]*

Pattern effects: *[Instance name; preserved / modified / extended / replaced / removed / newly introduced; reason]*

Refactoring: *[Real code smell, technique, diff and behavior-preservation evidence]*

Before/after metrics: *[Same measurement scope and tool; source LOC and actual test results; missing measures explained]*

3.1 Updated pattern counts

Add: For each snapshot record distinct pattern types, instances, concrete participant classes and rejected/uncertain observations. Keep the instance name when its collaboration continues. For replacement/new collaboration, name it and explain its relationship to the previous one.

Interpretation: *[Benefits, trade-offs and remaining limitations; more or fewer LOC alone does not prove better design]*

4 Part 3 Cross-language comparison

Add: For each confirmed pattern type, choose one representative verified instance. Implement its small problem/snippet in Java, Python, JavaScript and C++; preserve the same pattern and behavior.

Selected instances: *[Source instance names and pattern types; selection reasons]*

Common specifications: *[Inputs, outputs, rules, error cases and shared fixtures]*

Implementations: *[Paths, versions and run commands for all four languages]*

Actual results: *[Passed/total cases and logs per implementation; mark not run honestly]*

Static and dynamic comparison: *[Source LOC, maximum function cyclomatic complexity, runtime/compiler, test command/result, median process time and language notes; record in cross_language.csv]*

Comparison: *[Participant structures, abstraction, inheritance/composition and language features]*

Technology discussion: *[Short sourced discussion relevant to the course]*

5 LLM records and human verification

Add: Use free/publicly accessible tools. Save exact prompts, supplied inputs, outputs and verification notes for major analyses in each part. Do not submit invented LLM records as actual experiments.

Run and model: *[Displayed model, access tier, date and part]*

Inputs and outputs: *[Paths; source commit and supplied code/context]*

Claims and evaluation: *[Correct / partially correct / incorrect / cannot confirm; evidence, reviewer and correction]*

Generated code: *[What was used and how it was tested]*

6 Findings, contributions and references

Findings: *[Evidence-supported conclusions and limitations]*

Contributions: *[Roll number, name, batch and actual contribution; link to source/evidence]*

References: *[Application, pattern definitions, framework documents and tools]*

7 Team folder and datasets (take a screenshots from the GitHub repository— your team)

```
A1/
  README.md
  A1_Report.tex
  A1_Final_Report.pdf
  data/   patterns.csv, pattern_counts.csv,
          cross_language.csv, llm.csv
  Part1/  report PDF, code/, evidence/, llm/
  Part2/  report PDF, code/, evidence/, llm/
  Part3/  report PDF, code/, test-cases/, evidence/, llm/
```